

Trả lời câu hỏi 1:

1.1

Đánh giá Cohesion và God Class

- **Cohesion (Độ gắn kết): THẤP.**
 - Lý do: Class OrderService đang ôm đồm quá nhiều việc không liên quan đến nhau: Nghiệp vụ (tính tiền), Lưu trữ (database), Giao tiếp (email/sms), và Hiển thị (PDF).
- **God Class: CÓ.**
 - Nó biết quá nhiều và làm quá nhiều, dẫn đến việc sửa bất kỳ cái gì cũng phải đụng vào nó.

1.2

Các nguyên lý SOLID bị vi phạm:

1. S - Single Responsibility Principle (SRP):

- Vi phạm: OrderService có nhiều hơn một lý do để thay đổi.
 - Nếu muốn đổi cách in PDF -> phải sửa class này.
 - Nếu muốn đổi nội dung Email -> phải sửa class này.
 - Nếu đổi logic tính tiền -> phải sửa class này.
- Liên hệ Slide: Slide minh họa việc nhồi nhét quá nhiều trách nhiệm vào một class sẽ gây ra vấn đề "Too many responsibilities on a single thing can cause problems". Giống như class Customer trong slide vừa xử lý database vừa xử lý logging ,.

2. O - Open/Closed Principle (OCP):

- Vi phạm:
 - Khi muốn thêm Zalo Notification, bạn phải sửa code trong hàm gửi thông báo (thêm if-else).
 - Khi muốn thêm chiến lược giảm giá mới, bạn phải sửa hàm applyDiscount.
- Liên hệ Slide: Giống ví dụ getDiscount dùng if-else để kiểm tra loại khách hàng (_CustType == 1). Slide khuyên nên dùng kế thừa/interface để mở rộng tính năng mà không sửa code cũ.

3. D - Dependency Inversion Principle (DIP):

- Vi phạm:
 - `saveOrderToDatabase()` đang phụ thuộc trực tiếp vào MySQL. Khi đổi sang MongoDB, bạn phải đập đi xây lại hàm này.
 - Các hàm gửi tin nhắn đang phụ thuộc trực tiếp vào các provider cụ thể.
- Liên hệ Slide: Slide chỉ ra sai lầm khi class `Customer` phụ thuộc trực tiếp vào `FileLogger` (khởi tạo bằng `new`). Giải pháp đúng là phụ thuộc vào interface `ILogger` để có thể dễ dàng thay bằng `EmailLogger` hay `EventViewerLogger`,

1.3

ĐỀ XUẤT THIẾT KẾ MỚI

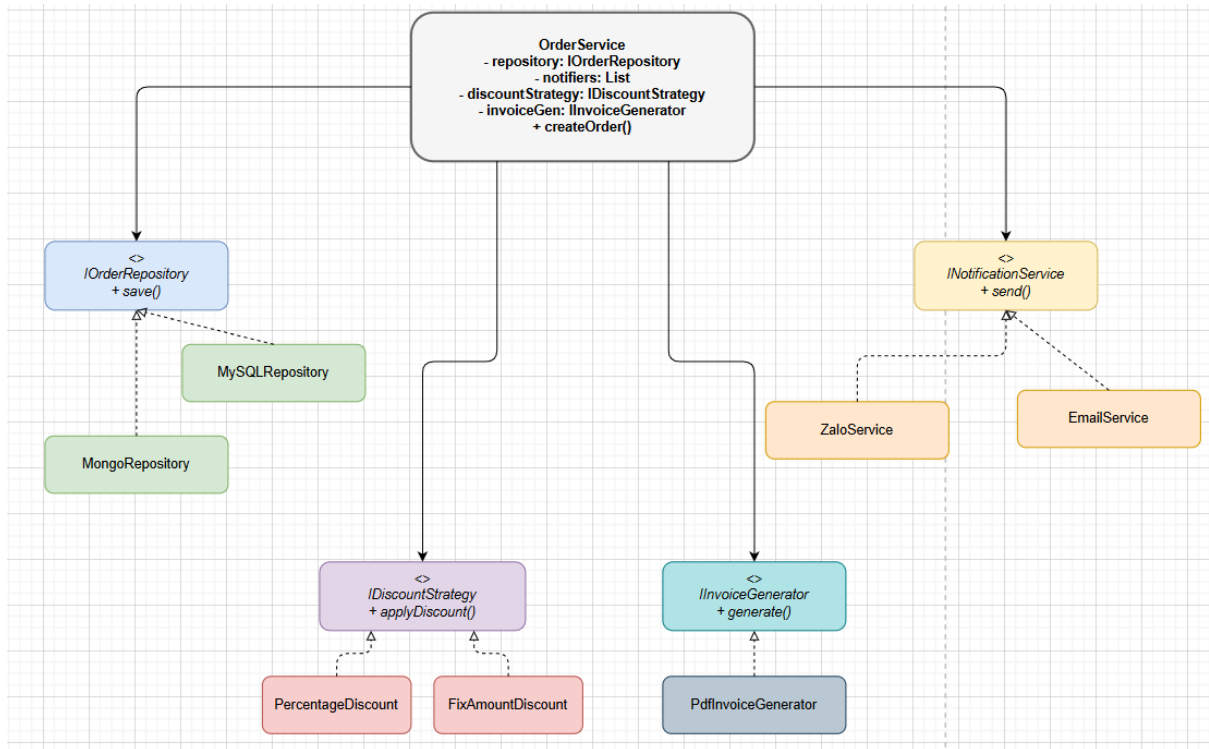
Để giải quyết, chúng ta sẽ "bẻ nhỏ" `OrderService` và sử dụng các Interface (Áp dụng DIP và Strategy Pattern cho OCP).

Cấu trúc Class/Module đề xuất:

1. Core Domain (Giữ logic chính):
 - `OrderService`: Chỉ làm nhiệm vụ điều phối luồng đi (Orchestrator).
 2. Abstraction (Interfaces - Các "Ổ cắm"):
 3. Implementation (Các class cụ thể - Các "Phích cắm"):
- *Database*: `MySQLOrderRepository`, `MongoOrderRepository` (Implement `IOrderRepository`).
 - *Notification*: `EmailService`, `SMSService`, `ZaloService` (Implement `INotificationService`).
 - *Discount*: `PercentageDiscount`, `FixAmountDiscount` (Implement `IDiscountStrategy`).
 - *Invoice*: `PdfInvoiceGenerator` (Implement `IIssueGenerator`).

1.4

Sơ đồ



Trả lời câu hỏi 2:

1. PHÂN TÍCH & TRẢ LỜI

Thiết kế hiện tại vi phạm nguyên lý nào?

- Vi phạm: Open/Closed Principle (OCP) – Nguyên lý Đóng/Mở.
- Phân tích dựa trên tài liệu:
 - Trong tài liệu (trang 10), ví dụ vi phạm OCP là class **Customer** sử dụng **if-else** để kiểm tra loại khách hàng (**_CustType**) và tính toán giảm giá.
 - Tương tự, nếu giữ thiết kế **PaymentService** hiện tại, khi muốn thêm ví điện tử hay **Crypto**, bạn sẽ phải:
 1. Hoặc viết thêm hàm mới (**payByMomo**, **payByCrypto**...).
 2. Hoặc sửa hàm **pay** cũ bằng cách chèn thêm **if-else** / **switch-case** để kiểm tra loại thanh toán.
 - Cả hai cách đều là sửa đổi (modification) code đang chạy ổn định -> Vi phạm OCP.

Hậu quả của việc sửa **PaymentService** liên tục

Nếu mỗi lần thêm phương thức thanh toán lại mở class PaymentService ra sửa:

1. Rủi ro lỗi (Regression Bugs): Bạn có thể vô tình làm hỏng logic thanh toán thẻ (Credit Card) đang chạy tốt khi đang mải viết code cho Crypto.
2. Khó bảo trì: Class PaymentService sẽ ngày càng phình to với hàng tá hàm xử lý cho từng loại thanh toán.
3. Tốn công kiểm thử (Testing): Mỗi lần sửa class này, bạn phải test lại toàn bộ các phương thức thanh toán, không chỉ phương thức mới thêm.

2. ĐỀ XUẤT THIẾT KẾ MỚI (Tuân thủ OCP)

Chúng ta sẽ sử dụng Interface (Trừu tượng hóa) để định nghĩa hành vi chung, giống như cách tài liệu đề xuất tách SilverCustomer và goldCustomer (trang 12).

Minh họa bằng Java:

Bước 1: Tạo Interface chung (Abstraction) Đây là "hợp đồng" mà mọi phương thức thanh toán phải tuân theo.

```
// Abstraction
```

```
public interface IPaymentStrategy {  
  
    void pay(double amount);  
  
}
```

Bước 2: Triển khai các phương thức cụ thể (Extension) Mỗi phương thức thanh toán là một class riêng biệt.

```
// Phương thức cũ
```

```
public class CreditCardPayment implements IPaymentStrategy {  
  
    public void pay(double amount) {  
  
        System.out.println("Thanh toán " + amount + " bằng Thẻ Tín Dụng");  
  
    }  
  
}
```

```
// Phương thức mới 1: Momo
```

```
public class MomoPayment implements IPaymentStrategy {  
  
    public void pay(double amount) {
```

```
        System.out.println("Thanh toán " + amount + " qua ví Momo");
    }
}
```

// Phương thức mới 2: Crypto

```
public class CryptoPayment implements IPaymentStrategy {
    public void pay(double amount) {
        System.out.println("Thanh toán " + amount + " bằng USDT");
    }
}
```

Bước 3: PaymentService chỉ gọi Interface (Closed for Modification)

```
public class PaymentService {
    // Chỉ phụ thuộc vào Interface, không phụ thuộc class cụ thể
    public void processPayment(IPaymentStrategy paymentMethod, double amount) {
        paymentMethod.pay(amount);
    }
}
```

3. GIẢI THÍCH: TẠI SAO GIÚP HỆ THỐNG OPEN - CLOSED?

- Open for Extension (Mở để mở rộng):
 - Khi sắp yêu cầu thêm thanh toán bằng ZaloPay, bạn chỉ cần tạo class mới ZaloPayPayment implements IPaymentStrategy.
 - Bạn đang mở rộng hệ thống bằng cách thêm code mới (new class).
- Closed for Modification (Đóng để sửa đổi):
 - Bạn không cần sửa bất kỳ dòng code nào trong class PaymentService. Hàm processPayment vẫn hoạt động đúng với class ZaloPayPayment mới vì nó chỉ quan tâm đến IPaymentStrategy.
 - Code cũ được bảo vệ an toàn tuyệt đối.

Trả lời câu hỏi 3

A. High Cohesion (Độ gắn kết cao) - "Chuyên môn hóa"

Định nghĩa thực tế: Là trạng thái mà một class/module chỉ tập trung giải quyết một vấn đề cụ thể và các thành phần bên trong nó có mối liên hệ mật thiết để phục vụ mục đích đó.

- Minh chứng từ Tình huống 1 (Food Delivery):
 - Low Cohesion (Hiện trạng): Class OrderService chứa rất nhiều logic: vừa tính toán business logic, vừa lo kết nối Database, vừa lo định dạng PDF. Các hàm này rời rạc về mặt chức năng nhưng lại bị nhét chung vào một chỗ.
 - High Cohesion (Mục tiêu): Khi tách logic in ấn ra InvoiceGenerator, logic lưu trữ ra OrderRepository. Lúc này OrderService có độ gắn kết cao vì nó chỉ tập trung vào việc "điều phối đơn hàng".
- Liên hệ Slide: Slide minh họa sự vi phạm Cohesion bằng hình ảnh con dao đa năng với quá nhiều lưỡi dao. Slide khẳng định: "Too many responsibilities on a single thing can cause problems" (Quá nhiều trách nhiệm trên một vật thể có thể gây ra vấn đề).

B. Low Coupling (Độ phụ thuộc thấp) - "Kết nối lỏng lẻo"

Định nghĩa thực tế: Là trạng thái mà các module không phụ thuộc trực tiếp vào "ruột gan" (chi tiết hiện thực) của nhau, mà chỉ giao tiếp qua các "hộp đồng" (Interface/Abstraction).

- Minh chứng từ Tình huống 2 (Payment):
 - Tight Coupling (Hiện trạng): PaymentService gọi trực tiếp các hàm xử lý thẻ. Nếu ngân hàng thay đổi API, PaymentService chết theo.
 - Low Coupling (Mục tiêu): PaymentService chỉ giao tiếp với IPaymentStrategy. Dù bên dưới là Momo, Crypto hay Visa, PaymentService không cần biết và không bị ảnh hưởng.
- Liên hệ Slide: Slide minh họa Low Coupling qua nguyên lý DIP, thay vì Customer phụ thuộc trực tiếp vào FileLogger (cứng nhắc), hãy để nó phụ thuộc vào ILogger (linh hoạt).

Vì sao đây là "Golden Goal" (Mục tiêu vàng) của thiết kế phần mềm?

Sự kết hợp giữa High Cohesion và Low Coupling được gọi là "Golden Goal" vì nó giải quyết triệt để các thách thức cốt lõi trong vòng đời phát triển phần mềm, được minh chứng qua các nguyên lý trong slide:

Lý do 1: Tối ưu hóa khả năng bảo trì (Maintainability)

- Vấn đề: Khi code có độ gắn kết thấp (Low Cohesion), một class làm quá nhiều việc sẽ trở nên cồng kềnh, khó đọc và khó sửa lỗi.
- Giải pháp từ Golden Goal: High Cohesion đảm bảo mỗi class là một chuyên gia trong lĩnh vực của nó. Khi xảy ra lỗi, lập trình viên biết chính xác vị trí cần sửa.
- Dẫn chứng Slide: Slide trích dẫn ngôn ngữ Tây Ban Nha: "It is not the load but the OVERLOAD that kills" (Không phải gánh nặng, mà sự quá tải mới giết chết bạn). High Cohesion giúp loại bỏ sự "quá tải" này, tuân thủ nguyên lý SRP.

Lý do 2: Tối đa hóa khả năng mở rộng (Extensibility)

- Vấn đề: Khi code bị phụ thuộc chặt (High Coupling), việc thêm tính năng mới thường yêu cầu sửa đổi code cũ, dẫn đến rủi ro hỏng hóc hệ thống (Regression bugs).
- Giải pháp từ Golden Goal: Low Coupling cho phép hệ thống mở rộng bằng cách thêm code mới (Extension) thay vì sửa code cũ (Modification).
- Dẫn chứng Slide: Slide sử dụng hình ảnh minh họa chiếc xe kéo được nối thêm toa mới với câu thoại "Did you know these IT guys copied my idea of extensibility!". Điều này phản ánh nguyên lý OCP: Mở để mở rộng nhưng đóng để sửa đổi.

Lý do 3: Tăng cường tính linh hoạt và kiểm thử (Flexibility & Testability)

- Vấn đề: Nếu các module gắn chặt vào nhau, ta không thể tách rời chúng để kiểm thử hoặc thay thế công nghệ (ví dụ đổi Database).
- Giải pháp từ Golden Goal: Low Coupling cho phép thay thế các thành phần ("plug and play") mà không làm gián đoạn hệ thống.
- Dẫn chứng Slide: Slide đưa ra ví dụ cụ thể về việc thay thế FileLogger bằng EmailLogger hoặc EventViewerLogger. Nhờ phụ thuộc vào Interface ILogger, hệ thống có thể chuyển đổi cơ chế ghi log mà không cần sửa class Customer.

Kết luận: "Golden Goal" không chỉ là đích đến lý thuyết mà là giải pháp thực tiễn để xây dựng phần mềm bền vững. High Cohesion giúp nội bộ module khỏe mạnh (dễ sửa), trong khi Low Coupling giúp mối quan hệ giữa các module linh hoạt (dễ thay đổi). Đây chính là nền tảng để đạt được kiến trúc phần mềm tốt (Good software architecture) mà bộ nguyên lý SOLID hướng tới

